

Vacation Planner Agent with crew AI

Summary: -

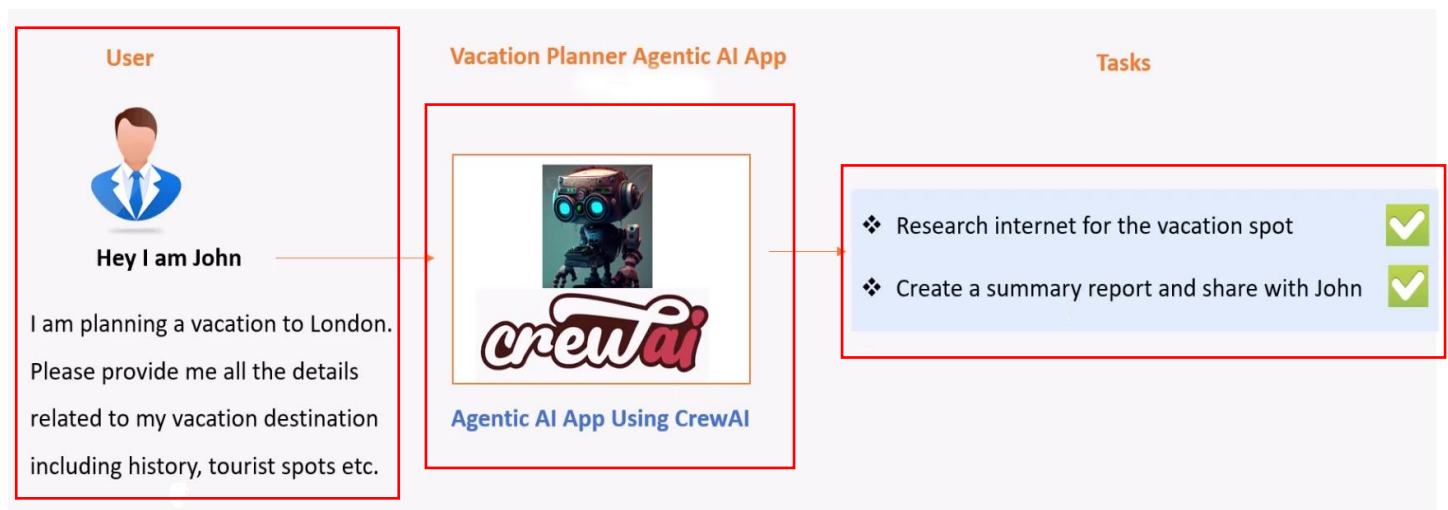
I developed an AI-powered Vacation Planner application using CrewAI, where multiple agents collaborate to generate travel plans. The system includes a research agent that gathers destination information using web tools (Serper API) and a planning agent that creates a detailed itinerary report. I configured AWS Bedrock for LLM integration, set up environment variables and dependencies, and implemented the project structure using Python and YAML configurations. Finally, I built a user-friendly interface using Streamlit to allow users to input destinations and receive complete vacation plans automatically.

Key Concept: -

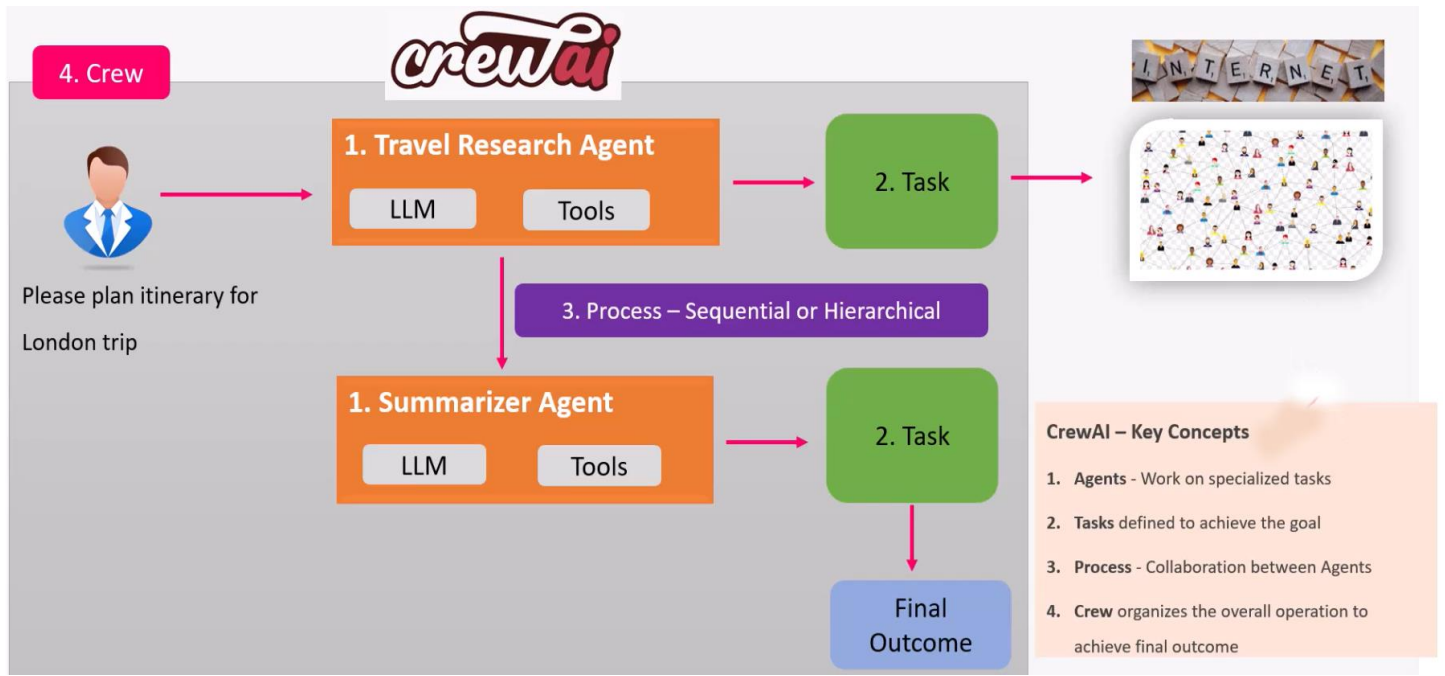
- **Crew AI: -**

CrewAI is an open-source Python framework that lets you create multiple AI agents, assign them roles, and make them work together as a “crew”

- Now let's continue with our vacation planning theme.



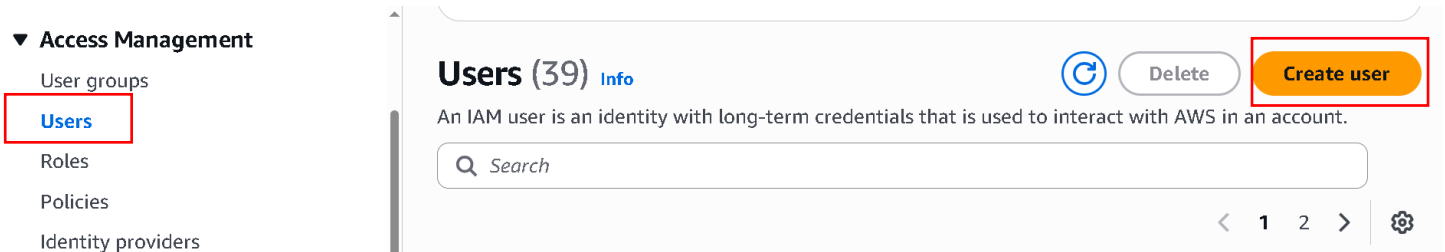
- And let's see we have a user, john.
- And john is planning a vacation to London, and he needs all the details related to the vacation destination, including history, tourist spots, and so on.
- Basically he wants a travel information for London.
- Now john can send this request to a vacation planner Agentic AI app built using Crew AI.
- Now this app will have two Agent.
- The first Agent should be able to research the internet for the vacation spot and in this case London.
- And the second Agent create a summary report and share with john.



- This is the work flow of our Agents.

Some PreRequisites before starting Creating App: -

- Install Vs code: - [Link](#)(windows)
- Install python(Recommended 3,11.9): - [Link](#)(windows)
- Install AWS CLI: - [Link](#)
- Now open Vs code and configure IAM user.
- So before you go to your VS Code open AWS console in your Browser.
- And search IAM and open it.



- Now in Navigation bar of IAM under Access Management click User.
- Then click Create User.

Specify user details

User details

User name

agent_ai

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

- ☐ Provide user access to the AWS Management Console - *optional*
In addition to console access, users with `SignInLocalDevelopmentAccess` permissions can use the same console credentials for programmatic access without the need for access keys.

i If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. [Learn more](#)

Cancel

Next

- Now Enter User name.
- And not tick the box and simply click Next.

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

- ☐ Add user to group
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.
- ☐ Copy permissions
Copy all group memberships, attached managed policies, and inline policies from an existing user.
- ☒ Attach policies directly
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

- Now in Set permissions select Attach policies directly.

Permissions policies (1/2441)



Create policy

Choose one or more policies to attach to your new user.

Filter by Type			
AdministratorAccess	X	All types	5 matches
☐ Policy name		Type	Attached entities
☒ + 📦 AdministratorAccess		AWS managed - job function	41
☐ + 📦 AdministratorAccess-A...		AWS managed	0
☐ + 📦 AdministratorAccess-A...		AWS managed	0

- Now in Permission Policies search and select AdministratorAccess.
- Now scroll down.

► Set permissions boundary - *optional*

Cancel

Previous

Next

- Now simply click next.

Tags - *optional*

Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Cancel

Previous

Create user

- Now review all thing and simply scroll down.
- And click Create user.

agent_ai [Info](#)

Delete

Summary

ARN

 arn:aws:iam::463646775279:user/agent_ai

Created

April 24, 2026, 14:38 (UTC+05:30)

Console access

Disabled

Last console sign-in

-

Access key 1

[Create access key](#)

Permissions

Groups

Tags

Security credentials

Last Accessed

- Now our user is created.
- Now go in Security credentials.

Access keys (0)

Create access key

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

No access keys. As a best practice, avoid using long-term credentials like access keys. Instead, use tools which provide short term credentials. [Learn more](#)

Create access key

- Now simply scroll down and search Access keys.
- Now in access keys click Create access key.

Access key best practices & alternatives [Info](#)

Avoid using long-term credentials like access keys to improve your security. Consider the following use cases and alternatives.

Use case



Command Line Interface (CLI)

You plan to use this access key to enable the AWS CLI to access your AWS account.



Local code

You plan to use this access key to enable application code in a local development environment to access your AWS account.



Application running on an AWS compute service

You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.

- Now simply select use case as CLI
- Then simply scroll down.

Confirmation



I understand the above recommendation and want to proceed to create an access key.

[Cancel](#)

[Next](#)

- Now tick the confirmation box and simply click Next.

Set description tag - *optional* [Info](#)

The description for this access key will be attached to this user as a tag and shown alongside the access key.

Description tag value

Describe the purpose of this access key and where it will be used. A good description will help you rotate this access key confidently later.

Maximum 256 characters. Allowed characters are letters, numbers, spaces representable in UTF-8, and: _ . : / = + - @

[Cancel](#)

[Previous](#)

[Create access key](#)

- Now Simply click Create Access Key.

Step 1

● Access key best practices & alternatives

Step 2 - optional

● Set description tag

Step 3

● Retrieve access keys

Retrieve access keys [Info](#)

Access key

If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key



AKIAWX44AK7X7PPA7LO5

Secret access key



***** [Show](#)

- Our Access Key is created.
- Now go to your VS Code that you open by Anaconda.
- And then create a Folder (like I'm created DEMO_APP)

```
EXPLORER      PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  CODE REFERENCE LOG
▼ DEMO_APP

PS C:\Users\Akash Kumar\Documents\Demo_app> python --version
Python 3.11.9
PS C:\Users\Akash Kumar\Documents\Demo_app> aws --version
aws-cli/2.34.36 Python/3.14.4 Windows/10 exe/AMD64
PS C:\Users\Akash Kumar\Documents\Demo_app> aws configure
AWS Access Key ID [*****7LO5]: AKIAWX44AK7X4FEV0QU7
AWS Secret Access Key [*****fiXc]: 0nbvpQuM9EAG16oWLY0LrV3g6YXQBbrG8t0rn9+7
Default region name [us-west-2]: us-west-2
Default output format [None]:
PS C:\Users\Akash Kumar\Documents\Demo_app> 
```

- Now firstly check python is install correctly or not by `python --version`
- Now check AWS cli is properly installed by `aws --version`
- Now all thing done then configure your Aws cli role in your VS Code
- Now write `aws configure`.
- Then they ask Access Key (copy access key from user that you created and paste it) then click Enter
- Now ask Secret Access Key (copy secret access key from user that you created and paste it) then click Enter.
- Then they ask region name: - give region in which you perform your all agent work
- In my case us-west-2
- Now they ask output format leave it and click Enter.
- Now configuration is done.

CrewAI Installation: -

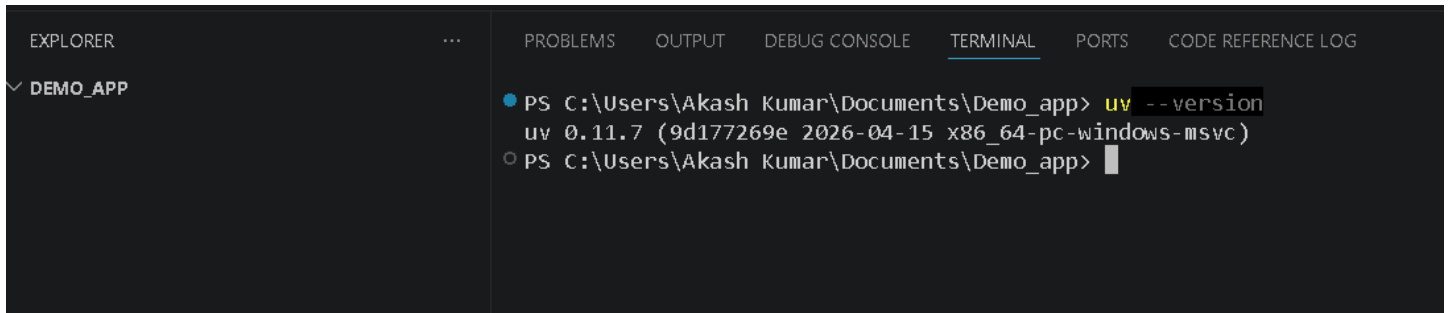
- pip install uv

```
EXPLORER      PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  CODE REFERENCE LOG
▼ DEMO_APP

PS C:\Users\Akash Kumar\Documents\Demo_app> pip install uv
Collecting uv
  Downloading uv-0.11.7-py3-none-win_amd64.whl.metadata (12 kB)
  Downloading uv-0.11.7-py3-none-win_amd64.whl (25.4 MB)
    25.4/25.4 MB 3.6 MB/s eta 0:00:00
Installing collected packages: uv
Successfully installed uv-0.11.7

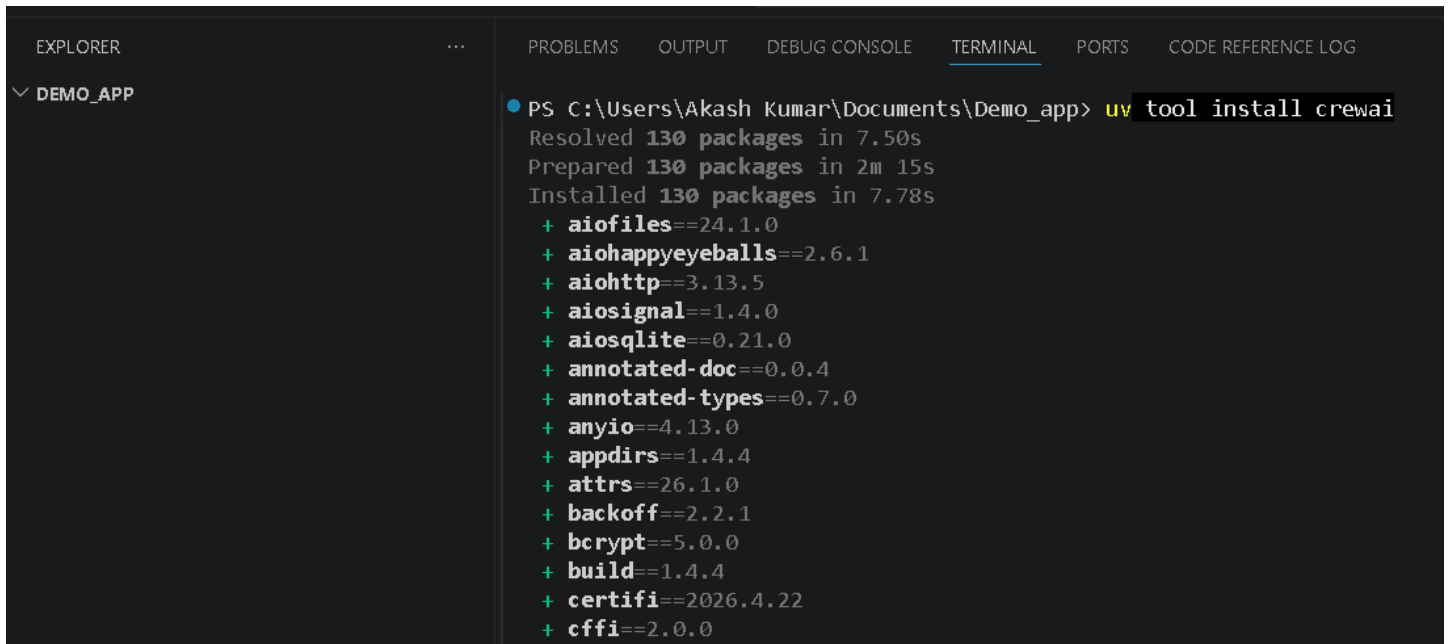
[notice] A new release of pip is available: 24.0 -> 26.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\Akash Kumar\Documents\Demo_app> 
```

- `uv --version`



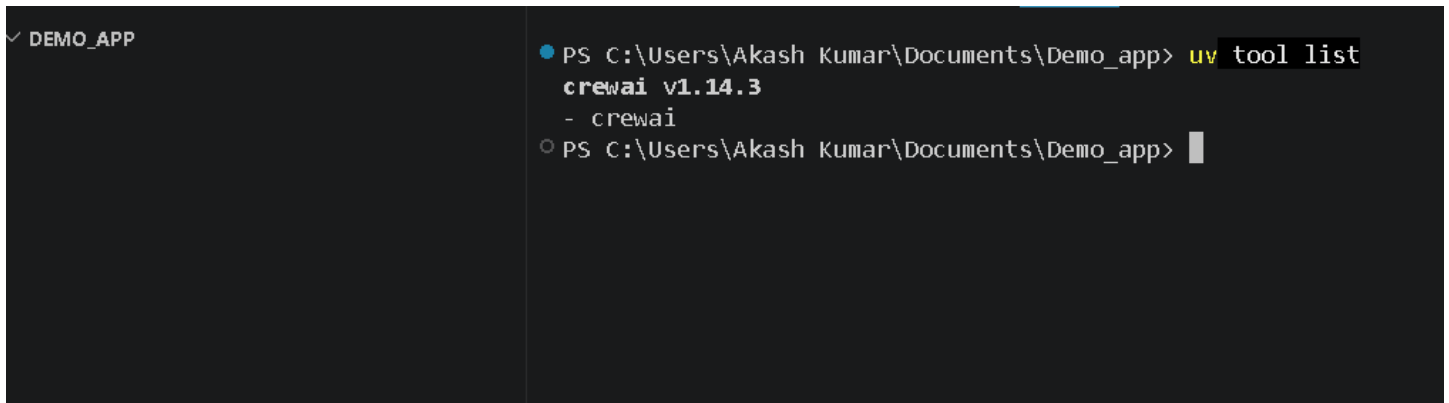
```
EXPLORER  ...  PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  CODE REFERENCE LOG
DEMO_APP
● PS C:\Users\Akash Kumar\Documents\Demo_app> uv --version
uv 0.11.7 (9d177269e 2026-04-15 x86_64-pc-windows-msvc)
○ PS C:\Users\Akash Kumar\Documents\Demo_app>
```

- `uv tool install crewai`



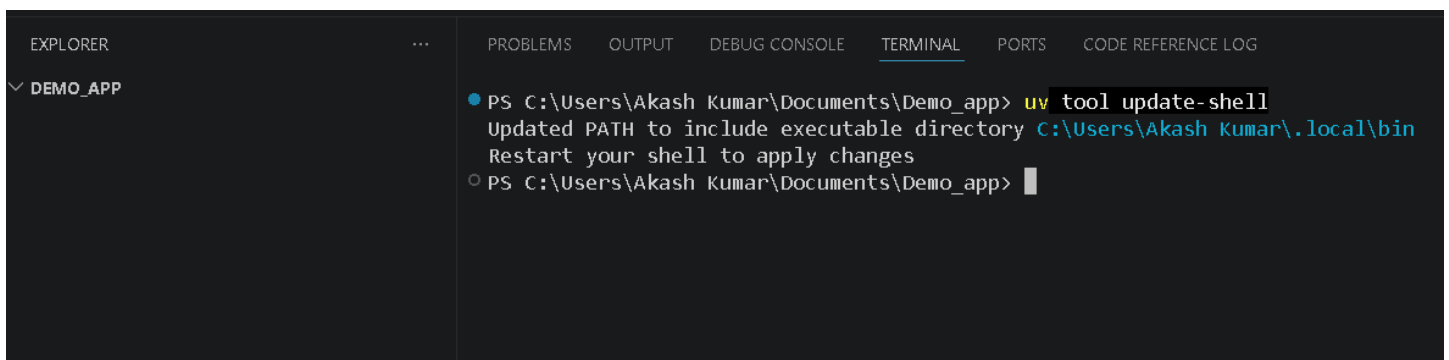
```
EXPLORER  ...  PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  CODE REFERENCE LOG
DEMO_APP
● PS C:\Users\Akash Kumar\Documents\Demo_app> uv tool install crewai
Resolved 130 packages in 7.50s
Prepared 130 packages in 2m 15s
Installed 130 packages in 7.78s
+ aiofiles==24.1.0
+ aiohappyeyeballs==2.6.1
+ aiohttp==3.13.5
+ aiosignal==1.4.0
+aiosqlite==0.21.0
+ annotated-doc==0.0.4
+ annotated-types==0.7.0
+ anyio==4.13.0
+ appdirs==1.4.4
+ attrs==26.1.0
+ backoff==2.2.1
+ bcrypt==5.0.0
+ build==1.4.4
+ certifi==2026.4.22
+ cffi==2.0.0
```

- `uv tool list`



```
EXPLORER  ...  PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  CODE REFERENCE LOG
DEMO_APP
● PS C:\Users\Akash Kumar\Documents\Demo_app> uv tool list
crewai v1.14.3
- crewai
○ PS C:\Users\Akash Kumar\Documents\Demo_app>
```

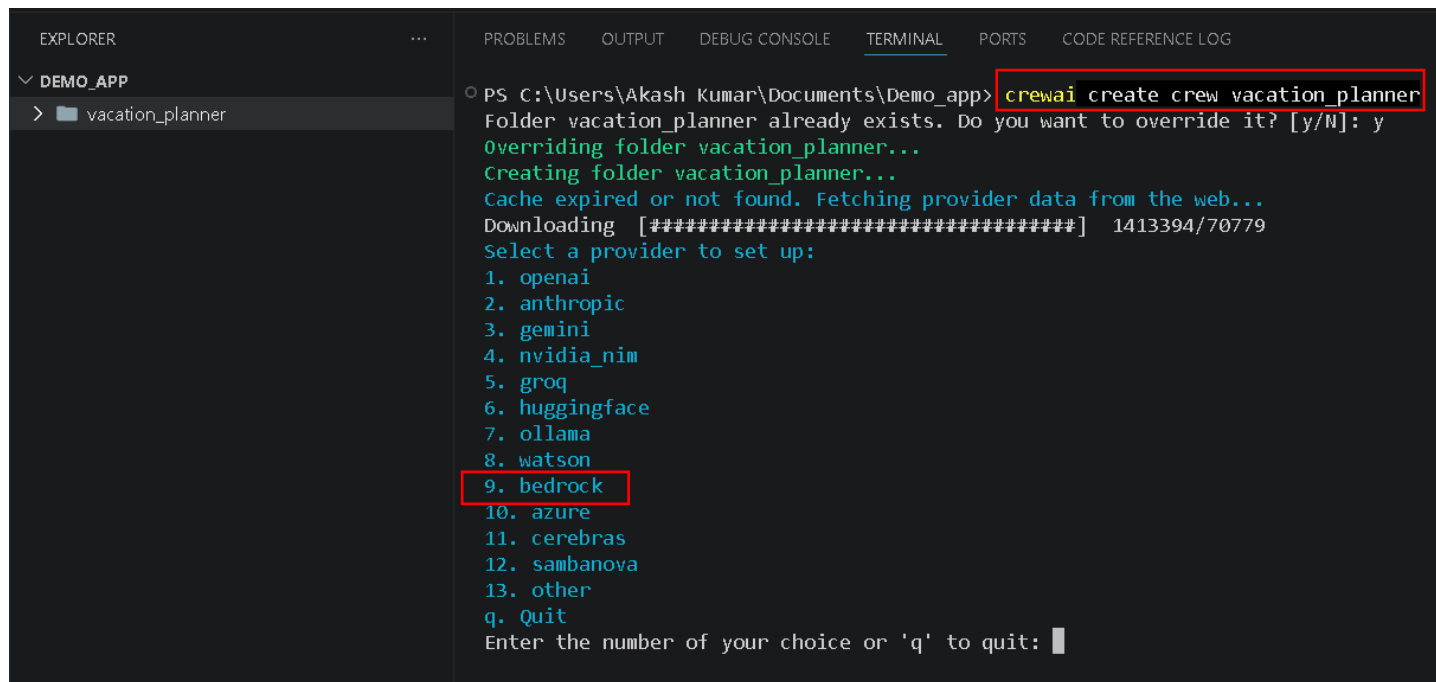
- `uv tool update-shell` (One time to set the path)



```
EXPLORER  ...  PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  CODE REFERENCE LOG
DEMO_APP
● PS C:\Users\Akash Kumar\Documents\Demo_app> uv tool update-shell
Updated PATH to include executable directory C:\Users\Akash Kumar\.local\bin
Restart your shell to apply changes
○ PS C:\Users\Akash Kumar\Documents\Demo_app>
```

Now Create Project: -

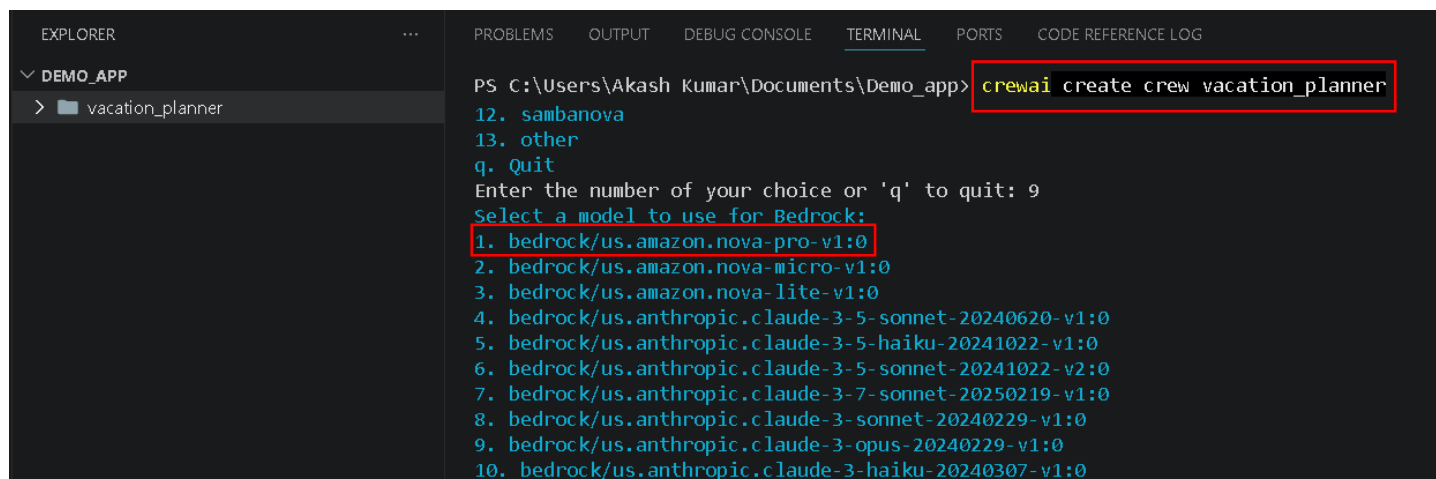
- crewai create crew vacation_planner



The screenshot shows the VS Code interface with the Explorer panel on the left showing a project named 'DEMO_APP' with a subfolder 'vacation_planner'. The Terminal panel on the right shows the command 'crewai create crew vacation_planner' being executed. The output indicates that the folder 'vacation_planner' already exists and is being overridden. It then proceeds to fetch provider data from the web and displays a list of providers to set up. The list includes: 1. openai, 2. anthropic, 3. gemini, 4. nvidia_nim, 5. groq, 6. huggingface, 7. ollama, 8. watson, 9. bedrock, 10. azure, 11. cerebras, 12. sambanova, 13. other, and q. Quit. The option '9. bedrock' is highlighted with a red box.

```
PS C:\Users\Akash Kumar\Documents\Demo_app> crewai create crew vacation_planner
Folder vacation_planner already exists. Do you want to override it? [y/N]: y
Overriding folder vacation_planner...
Creating folder vacation_planner...
Cache expired or not found. Fetching provider data from the web...
Downloading [#####] 1413394/70779
Select a provider to set up:
1. openai
2. anthropic
3. gemini
4. nvidia_nim
5. groq
6. huggingface
7. ollama
8. watson
9. bedrock
10. azure
11. cerebras
12. sambanova
13. other
q. Quit
Enter the number of your choice or 'q' to quit: █
```

- select 9(bedrock)



The screenshot shows the VS Code interface with the Explorer panel on the left showing a project named 'DEMO_APP' with a subfolder 'vacation_planner'. The Terminal panel on the right shows the command 'crewai create crew vacation_planner' being executed. The output indicates that the folder 'vacation_planner' already exists and is being overridden. It then proceeds to fetch provider data from the web and displays a list of providers to set up. The list includes: 12. sambanova, 13. other, q. Quit. The option '9. bedrock' is highlighted with a red box. The terminal then prompts 'Enter the number of your choice or 'q' to quit: 9'. It then prompts 'Select a model to use for Bedrock:'. The list of models includes: 1. bedrock/us.amazon.nova-pro-v1:0, 2. bedrock/us.amazon.nova-micro-v1:0, 3. bedrock/us.amazon.nova-lite-v1:0, 4. bedrock/us.anthropic.claude-3-5-sonnet-20240620-v1:0, 5. bedrock/us.anthropic.claude-3-5-haiku-20241022-v1:0, 6. bedrock/us.anthropic.claude-3-5-sonnet-20241022-v2:0, 7. bedrock/us.anthropic.claude-3-7-sonnet-20250219-v1:0, 8. bedrock/us.anthropic.claude-3-sonnet-20240229-v1:0, 9. bedrock/us.anthropic.claude-3-opus-20240229-v1:0, and 10. bedrock/us.anthropic.claude-3-haiku-20240307-v1:0. The option '1. bedrock/us.amazon.nova-pro-v1:0' is highlighted with a red box.

```
PS C:\Users\Akash Kumar\Documents\Demo_app> crewai create crew vacation_planner
12. sambanova
13. other
q. Quit
Enter the number of your choice or 'q' to quit: 9
Select a model to use for Bedrock:
1. bedrock/us.amazon.nova-pro-v1:0
2. bedrock/us.amazon.nova-micro-v1:0
3. bedrock/us.amazon.nova-lite-v1:0
4. bedrock/us.anthropic.claude-3-5-sonnet-20240620-v1:0
5. bedrock/us.anthropic.claude-3-5-haiku-20241022-v1:0
6. bedrock/us.anthropic.claude-3-5-sonnet-20241022-v2:0
7. bedrock/us.anthropic.claude-3-7-sonnet-20250219-v1:0
8. bedrock/us.anthropic.claude-3-sonnet-20240229-v1:0
9. bedrock/us.anthropic.claude-3-opus-20240229-v1:0
10. bedrock/us.anthropic.claude-3-haiku-20240307-v1:0
```

- Now select amazon.nova.pro
- So, scroll down and Enter 1.


```

q. Quit
Enter the number of your choice or 'q' to quit: 1
Enter your AWS Access Key ID (press Enter to skip):
Enter your AWS Secret Access Key (press Enter to skip):
Enter your AWS Region Name (press Enter to skip):
API keys and model saved to .env file
Selected model: bedrock/us.amazon.nova-pro-v1:0
- Created vacation_planner\.gitignore
- Created vacation_planner\pyproject.toml
- Created vacation_planner\README.md
- Created vacation_planner\knowledge\user_preference.txt
- Created vacation_planner\src\vacation_planner\__init__.py
- Created vacation_planner\src\vacation_planner\main.py
- Created vacation_planner\src\vacation_planner\crew.py
- Created vacation_planner\src\vacation_planner\tools\custom_tool.py
- Created vacation_planner\src\vacation_planner\tools\__init__.py
- Created vacation_planner\src\vacation_planner\config\agents.yaml
- Created vacation_planner\src\vacation_planner\config\tasks.yaml
Crew vacation_planner created successfully!
PS C:\Users\Akash Kumar\Documents\Demo_app>

```

- Now you get option to Enter Access Key, secret Access Key, Region.
- Simply Press Enter and Skip this part because we already configure our AWS cli in starting.
- Now you see in this Green it's created basically it's called project scaffolding.

```

EXPLORER
  DEMO_APP
    vacation_planner
      knowledge
        user_preference.txt
      src\vacation_planner
        config
          agents.yaml
          tasks.yaml
        tools
          __init__.py
          crew.py
          main.py
        tests
      .env
      .gitignore
      AGENTS.md
      pyproject.toml
      README.md

agents.yaml
vacation_planner > src > vacation_planner > config > agents.yaml
1  researcher:
2    role: >
3      {topic} Senior Data Researcher
4    goal: >
5      Uncover cutting-edge developments in {topic}
6    backstory: >
7      You're a seasoned researcher with a knack for uncovering the latest
8      developments in {topic}. Known for your ability to find the most relevant
9      information and present it in a clear and concise manner.
10
11  reporting_analyst:
12    role: >
13      {topic} Reporting Analyst
14    goal: >
15      Create detailed reports based on {topic} data analysis and research findings
16    backstory: >
17      You're a meticulous analyst with a keen eye for detail. You're known for
18      your ability to turn complex data into clear and concise reports, making
19      it easy for others to understand and act on the information you provide.

```

- Now from Previous commands some files is created in your folder DEMO_APP like vacation_planner.
- Now go inside it and then go inside src.
- In src go inside config and open agent.yaml file .
- Now replace previous written YAML code from this code.

Configuration for the agents used in the vacation planner project.

vacation_researcher:

role: >

Senior Vacation Researcher

goal: >

Research and find the best vacation spots related to {topic}.

backstory: >

You are a seasoned vacation researcher, known for your ability to discover hidden gems and unique experiences.

Your goal is to find unforgettable destinations tailored to a variety of travel interests.

itinerary_planner:

role: >

Itinerary Planner

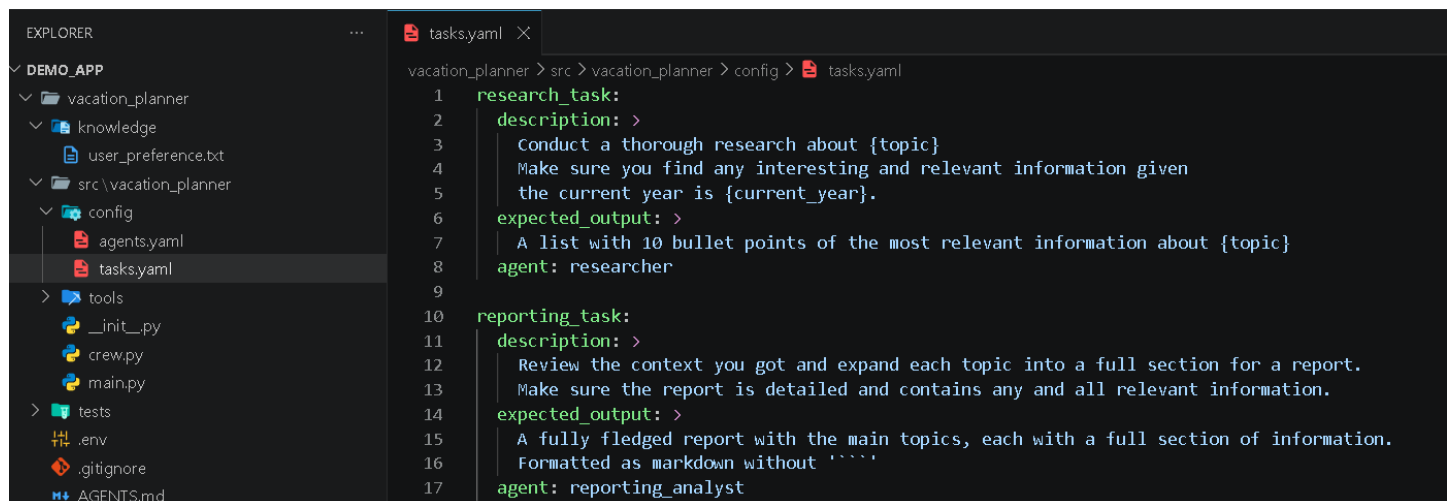
goal: >

Create a detailed itinerary highlighting the best tourist spots in {topic}, including history of the city, list of must-try local foods.

backstory: >

You are a meticulous planner with a keen eye for detail.

You specialize in crafting organized travel schedules and suggesting local culinary experiences to make vacations truly memorable.



```
EXPLORER
DEMO_APP
├── vacation_planner
│   ├── knowledge
│   │   └── user_preference.txt
│   └── src \ vacation_planner
│       ├── config
│       │   ├── agents.yaml
│       │   └── tasks.yaml
│       ├── tools
│       │   ├── _init_.py
│       │   ├── crew.py
│       │   └── main.py
│       ├── tests
│       ├── .env
│       ├── .gitignore
│       └── AGENTS.md
tasks.yaml
vacation_planner > src > vacation_planner > config > tasks.yaml
1  research_task:
2  description: >
3      Conduct a thorough research about {topic}
4      Make sure you find any interesting and relevant information given
5      the current year is {current_year}.
6  expected_output: >
7      A list with 10 bullet points of the most relevant information about {topic}
8  agent: researcher
9
10 reporting_task:
11 description: >
12     Review the context you got and expand each topic into a full section for a report.
13     Make sure the report is detailed and contains any and all relevant information.
14 expected_output: >
15     A fully fledged report with the main topics, each with a full section of information.
16     Formatted as markdown without '```'
17 agent: reporting_analyst
```

- Now open the task.yaml present in same folder.
- And replace the previous code with this.

Configuration for tasks in the vacation planner project.

research_task:

description: >

Conduct thorough research about the travel destination {topic}.

Find interesting and relevant information to enrich the travel experience.

expected_output: >

A list of 15 bullet points highlighting the most important and exciting facts about {topic}.

agent: vacation_researcher

reporting_task:

description: >

Review the research findings and create a detailed report about {topic}.

expected_output: >

A fully fledged report summarizing key information about {topic},

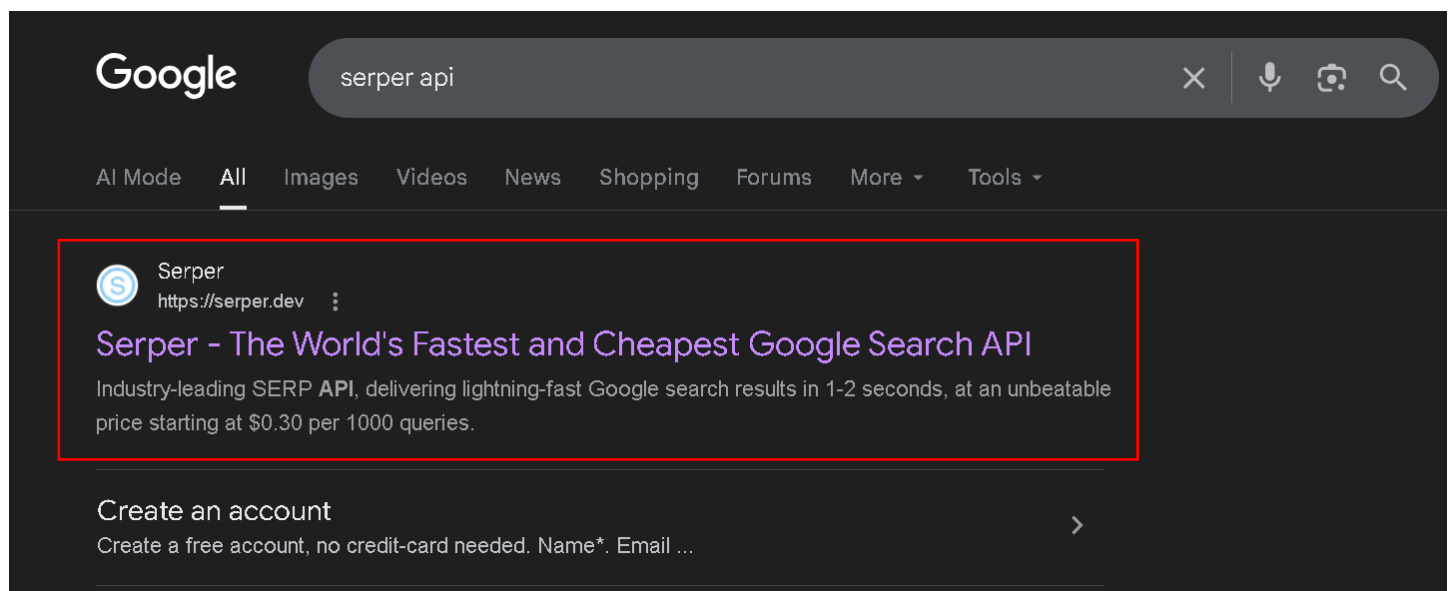
formatted in Markdown without using code block syntax (do not use triple backticks).

agent: itinerary_planner

output_file: report.md

output_format: markdown

- Add Tools to Agents in Crew.py
- Now we provide our agent, researcher agent tool which allow it to search the internet.
- So there are lot of tools but we use Google serper search.
- And for this you need a API key.
- So just we need to go on the Browser and search serper Api.



- Click on first Link.

The World's Fastest & Cheapest Google Search API

- Simply click on signup button.

Create an account

Create a free account, no credit-card needed.

Name *

First

Last

Email *

Password *

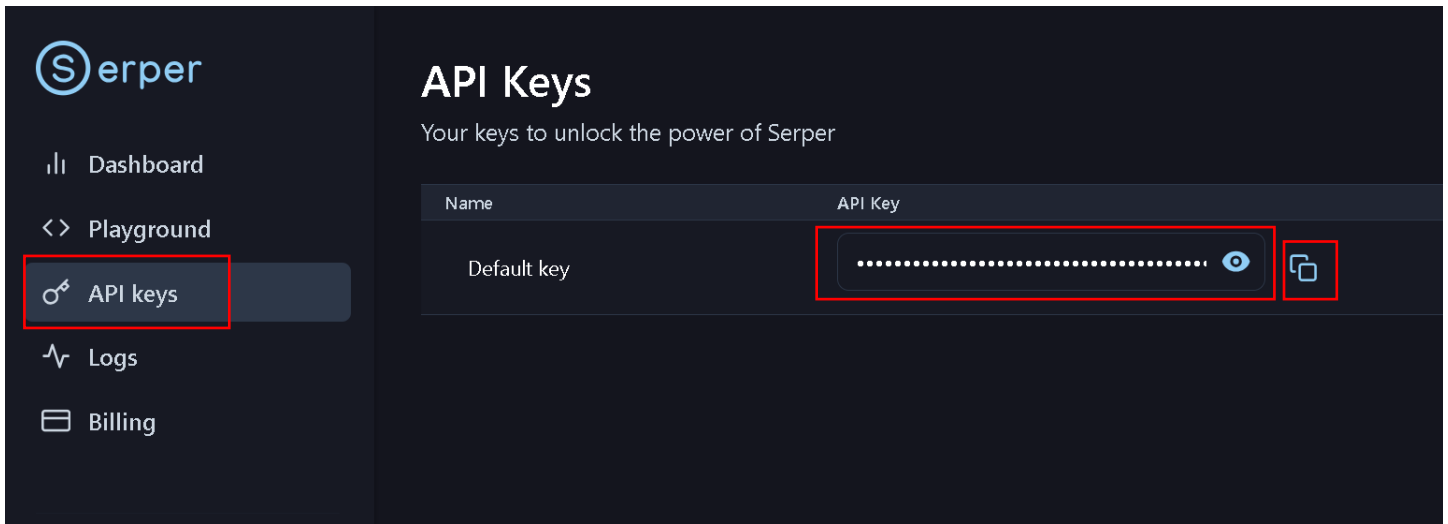


Success!

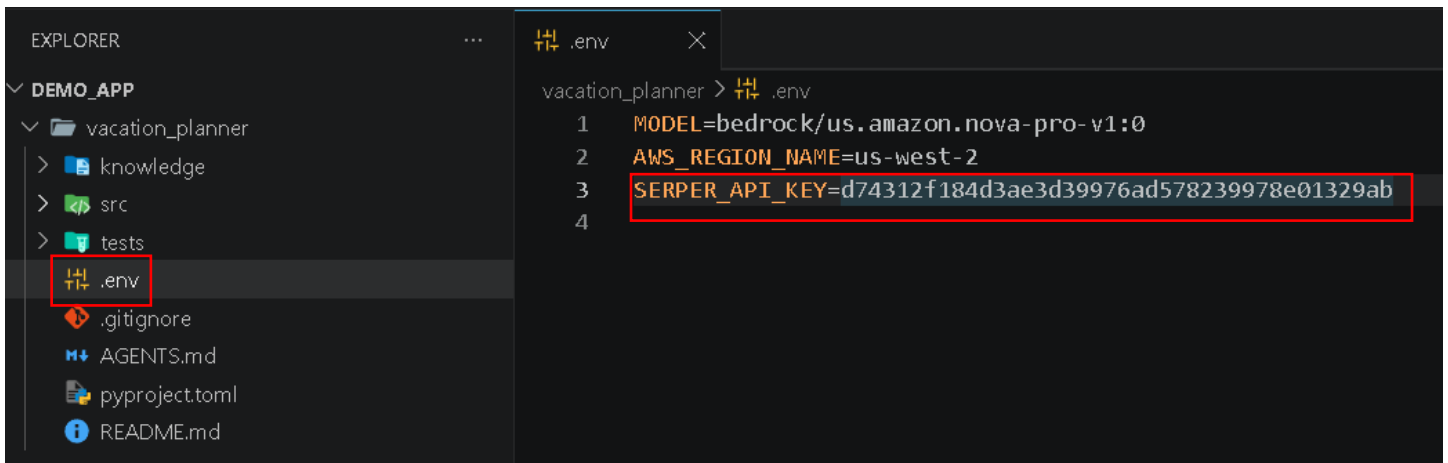


Create account

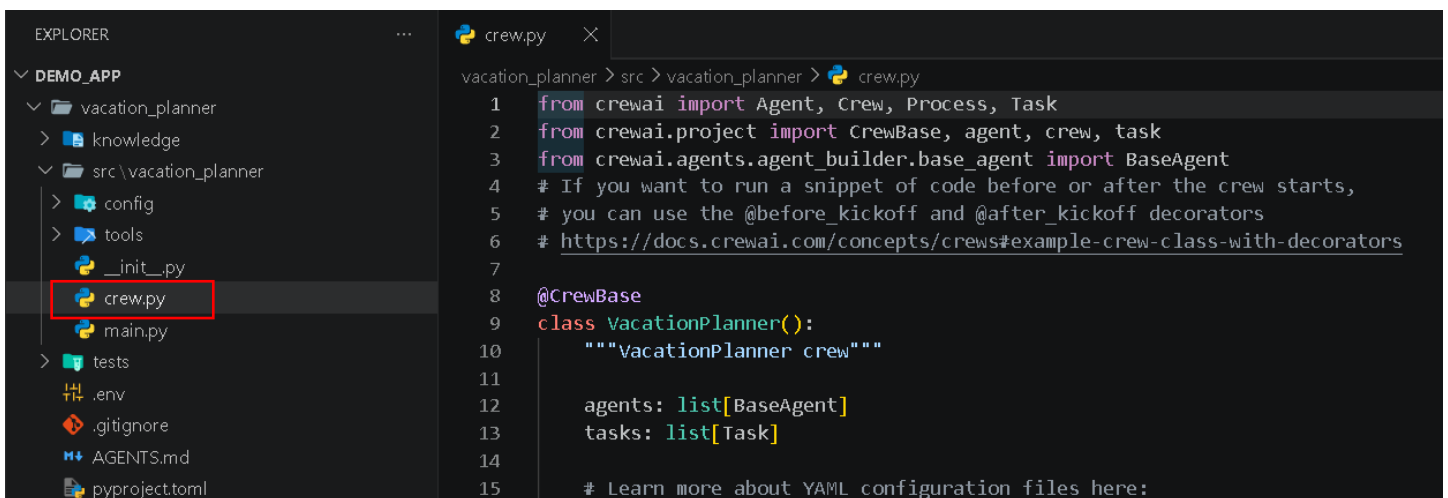
- Now Enter Details and click Create account.
- Now simply go to your Mail and complete verification.
- And then try to login.



- Now your account is created and login successfully to your account.
- Now In Navigation bar click API keys.
- And then simply copy it.
- Now go to your VS Code.



- Now open .env file inside in vacation_planner.
- And add SERPER_API_KEY = paste your API that you copied from serper website.



- Now click src.
- In src click vacation_planner.
- In vacation_planner open crew.py

```
crew.py 4 X agents.yaml
vacation_planner > src > vacation_planner > crew.py > VacationPlanner > vacation_researcher
1 from crewai import Agent, Crew, Process, Task
2 from crewai.project import CrewBase, agent, crew, task
3 from crewai.agents.agent_builder.base_agent import BaseAgent
4 from crewai.tools import SerperDevTool
5 # If you want to run a snippet of code before or after the crew starts,
6 # you can use the @before_kickoff and @after_kickoff decorators
7 # https://docs.crewai.com/concepts/crews#example-crew-class-with-decorators
8
9 @CrewBase
10 class VacationPlanner():
11     """VacationPlanner crew"""
12
```

- Now import SerperDevTool by “from crewai tools import SerperDevTool”
- Now one thing we need to do is that define from where they will take API key.
- And we know that we define our API key in .env file.
- Basically we need to import one more thing and write one line code.

```
crew.py 4 X .env agents.yaml
vacation_planner > src > vacation_planner > crew.py > ...
1 from crewai import Agent, Crew, Process, Task
2 from crewai.project import CrewBase, agent, crew, task
3 from crewai.agents.agent_builder.base_agent import BaseAgent
4 from crewai.tools import SerperDevTool
5 import os
6
7 #initialize SerperDevTool
8 serper_dev_tool = SerperDevTool(api_key=os.environ.get("SERPER_API_KEY"))
9
10
```

- Write these two code so they can access API key: -
- Import os
- Serper_dev_tool = SerperDevTool(api_key=os.environ.get(“SERPER_API_KEY”))

```
27 # https://docs.crewai.com/concepts/agents#agent-tools
28 @agent
29 def vacation_researcher(self) -> Agent:
30     return Agent(
31         config=self.agents_config['vacation_researcher'], # type: ignore[index]
32         verbose=True,
33         tools=[serper_dev_tool],
34     )
```

- Now in first agent decorator(@agent).
- Copy first agent name from agents.yaml file.
- In agents.yaml first agent name that will search on internet is vacation_researcher
- copy this name from agents.yaml file and paste.
- Then add tools = [serper_dev_tool]

```

36 @agent
37 def itinerary_planner(self) -> Agent:
38     return Agent(
39         config=self.agents_config['itinerary_planner'], # type: ignore[index]
40         verbose=True
41     )
42

```

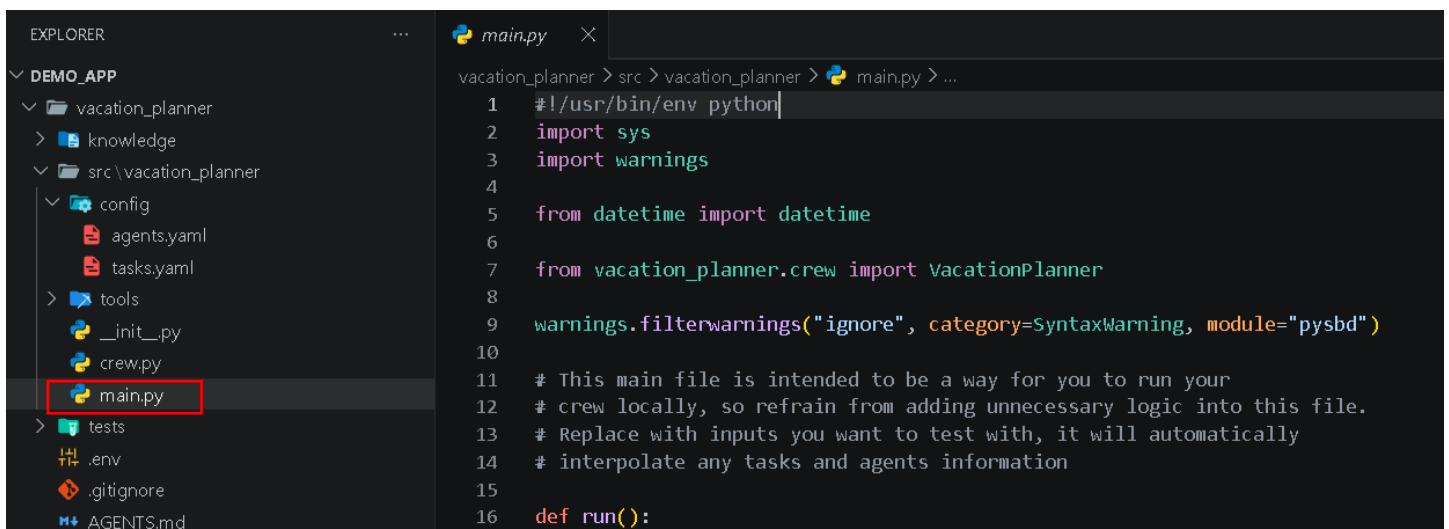
- Now scroll down and you see 2nd agent decorator(@agent).
- And change its name with 2nd agent name that will present in agents.yaml file.
- Now in our case 2nd agent name is itinerary_planner
- Now replace it with
- Now scroll down.

```

46 @task
47 def research_task(self) -> Task:
48     return Task(
49         config=self.tasks_config['research_task'], # type: ignore[index]
50     )
51
52 @task
53 def reporting_task(self) -> Task:
54     return Task(
55         config=self.tasks_config['reporting_task'], # type: ignore[index]
56         output_file='report.md'
57     )

```

- Now check tasks name from tasks.yaml file
- In our case we created two task in tasks.yaml file.
- So make sure in code task name is same as tasks.yaml file.
- Now all work donw in this file save this file,



```

EXPLORER
DEMO_APP
├── vacation_planner
│   ├── knowledge
│   └── src\vacation_planner
│       ├── config
│       │   ├── agents.yaml
│       │   └── tasks.yaml
│       ├── tools
│       ├── __init__.py
│       ├── crew.py
│       └── main.py
├── tests
├── .env
├── .gitignore
└── AGENTS.md

main.py
vacation_planner > src > vacation_planner > main.py > ...
1  #!/usr/bin/env python
2  import sys
3  import warnings
4
5  from datetime import datetime
6
7  from vacation_planner.crew import VacationPlanner
8
9  warnings.filterwarnings("ignore", category=SyntaxWarning, module="pysbd")
10
11  # This main file is intended to be a way for you to run your
12  # crew locally, so refrain from adding unnecessary logic into this file.
13  # Replace with inputs you want to test with, it will automatically
14  # interpolate any tasks and agents information
15
16  def run():

```

- Now in same src, vacation_planner file.
- Open main.py file.

```

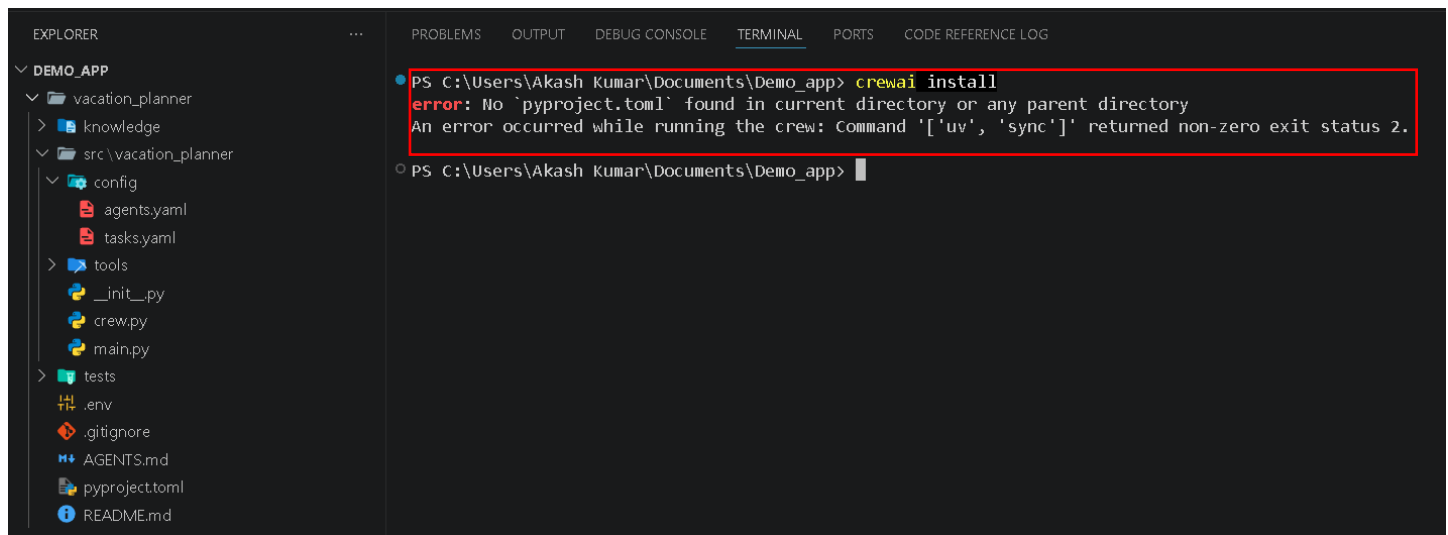
16 def run():
17     """
18     Run the crew.
19     """
20     inputs = {
21         'topic': 'London',
22         'current_year': str(datetime.now().year)
23     }
24

```

- Now scroll down and in run function.
- input object topic = place your want
- In our case our user want to know about London.
- So we give London.
- And save this file.

Now run the Project: -

- Open Terminal in VS Code.



```

EXPLORER
DEMO_APP
├── vacation_planner
│   ├── knowledge
│   └── src
│       └── vacation_planner
│           ├── config
│           │   ├── agents.yaml
│           │   └── tasks.yaml
│           ├── tools
│           │   ├── __init__.py
│           │   ├── crew.py
│           │   └── main.py
│           └── tests
├── .env
├── .gitignore
├── AGENTS.md
├── pyproject.toml
└── README.md

TERMINAL
PS C:\Users\Akash Kumar\Documents\Demo_app> crewai install
error: No 'pyproject.toml' found in current directory or any parent directory
An error occurred while running the crew: Command '['uv', 'sync']' returned non-zero exit status 2.
PS C:\Users\Akash Kumar\Documents\Demo_app>

```

- Write crewai install
- Now you see it's give error.
- Because we are not in correct folder we are in Demo_app.
- But we need to go inside Demo_app and then inside vacation_planner that present in Demo_app.
- So simply we write cd vacation_planner
- And then lock the dependencies by this code crewai install.


```
EXPLORER
DEMO_APP
├── vacation_planner
├── .venv
├── knowledge
├── src\vacation_planner
├── tests
├── .env
├── .gitignore
├── AGENTS.md
├── pyproject.toml
├── README.md
└── uv.lock

TERMINAL
PS C:\Users\Akash Kumar\Documents\Demo_app\vacation_planner> crewai install
Using CPython 3.11.9 interpreter at: C:\Users\Akash Kumar\AppData\Local\Programs\Python\Python311\python.exe
Creating virtual environment at: .venv
Resolved 148 packages in 5.38s
Built vacation-planner @ file:///C:/Users/Akash%20Kumar/Documents/Demo_app/vacation_planner
lance-namespace ----- 0 B/12.07 KiB
backoff ----- 0 B/14.79 KiB
Installed 141 packages in 29.46s
+ aiofiles==24.1.0
+ aiohappyeyeballs==2.6.1
+ aiohttp==3.13.5
+ aiosignal==1.4.0
+ aiosqlite==0.21.0
+ annotated-doc==0.0.4
+ annotated-types==0.7.0
+ anyio==4.13.0
+ appdirs==1.4.4
+ attrs==26.1.0
+ backoff==2.2.1
+ bcrypt==5.0.0
+ beautifulsoup4==4.13.5
+ build==1.4.4
+ certifi==2026.4.22
```

- Now dependencies is locked and some new file also added.
- You see in left navigation bar you see .venv file and uvlock.
- Now we need to install boto3 because its required for Amazon Bedrock.

```
PS C:\Users\Akash Kumar\Documents\Demo_app\vacation_planner> uv add boto3
Resolved 152 packages in 1.58s
Built vacation-planner @ file:///C:/Users/Akash%20Kumar/Documents/Demo_app/vacation_planner
Prepared 5 packages in 4.10s
Uninstalled 1 package in 31ms
Installed 5 packages in 3.42s
+ boto3==1.42.96
+ botocore==1.42.96
+ jmespath==1.1.0
+ s3transfer==0.16.1
~ vacation-planner==0.1.0 (from file:///C:/Users/Akash%20Kumar/Documents/Demo_app/vacation_planner)
PS C:\Users\Akash Kumar\Documents\Demo_app\vacation_planner>
```

- Enter “ uv add boto3 ”
- Our boto3 is installed.
- Now all thing done run the app by “ crewai run ”

```
PS C:\Users\Akash Kumar\Documents\Demo_app\vacation_planner> crewai run
Running the crew
```

 Crew Execution Started

Crew Execution Started

Name: VacationPlanner

ID: df40e067-8bf3-4290-b830-0789c61a8060

 Task Started

Task Started

Name: research_task

ID: 90b177cb-77af-41fa-9d5f-404fedb987a3

 Agent Started

159 x 41

Agent: **Senior Vacation Researcher**

Task: Conduct thorough research about the travel destination London. Find interesting and relevant information to enrich the travel experience.

 Tool Execution Started (#1)

Tool: **search_the_internet_with_serper**

Args: {'search_query': 'interesting and exciting facts about London'}

- You see first vacation planner Execution is started that search on Internet and give result using tool serper.

 Agent Final Answer

Agent: **Senior Vacation Researcher**

Final Answer:

- London Underground, opened in 1863, is the world's oldest subway system.
- Three of the top ten museums and galleries in the world are in London, with a total of 857 art galleries.
- London has four UNESCO World Heritage sites: Tower of London, Westminster, Greenwich, and Maritime Greenwich.
- The City of London is the smallest city in the UK.
- London has 20 secret underground rivers.
- The Knowledge of London test is one of the hardest taxi knowledge tests in the world.
- London is globally renowned for its rich history, vibrant culture, and landmarks like Tower Bridge, the Tower of London, Buckingham Palace, and the London Eye.
- London is full of rolling green parks, cherry red buses, royal palaces, and is a multicultural city with diverse attractions.
- London has ancient Roman roots and is home to the capital's oldest hat shop.
- London's smallest public statue is of a mouse.
- London is the world's largest financial center.
- There are 13 ghosts haunting one of London's spookiest locations.
- It is impossible for most people to lick their own elbow.
- A crocodile cannot stick its tongue out.
- A shrimp's heart is in its head.

 Task Completion

Task Completed

Name: research_task

Agent: Senior Vacation Researcher

- Now you see 1st agent research on internet and gives result.
- 1st agent task is completed.

Task Started

Task Started

Name: `reporting_task`

ID: `020d9c25-7e4a-43ea-b4e7-c0e04ed9416b`

Agent Started

Agent: **Itinerary Planner**

Task: Review the research findings and create a detailed report about London.

- Now second agent and second task is executing.
- Basically it will review first agent result and give detailed report.

Agent Final Answer

Agent: **Itinerary Planner**

Final Answer:

London: A Comprehensive Guide

Introduction

London, the capital city of the United Kingdom, is a vibrant metropolis with a rich tapestry of history, culture, and modern attractions. It is a city that effortlessly blends the old with the new, offering visitors a unique experience at every turn. From its ancient Roman roots to its status as the world's largest financial center, London is a city of contrasts and surprises.

Historical Overview

London's history dates back to Roman times when it was known as Londinium. Over the centuries, it has evolved into a global city, playing a pivotal role in world events. The city is home to numerous historical landmarks, including the Tower of London, Buckingham Palace, and the Houses of Parliament.

Key Attractions

Museums and Galleries

London boasts three of the top ten museums and galleries in the world, with a total of 857 art galleries. Notable institutions include:

- The British Museum
- The Tate Modern
- The National Gallery

UNESCO World Heritage Sites

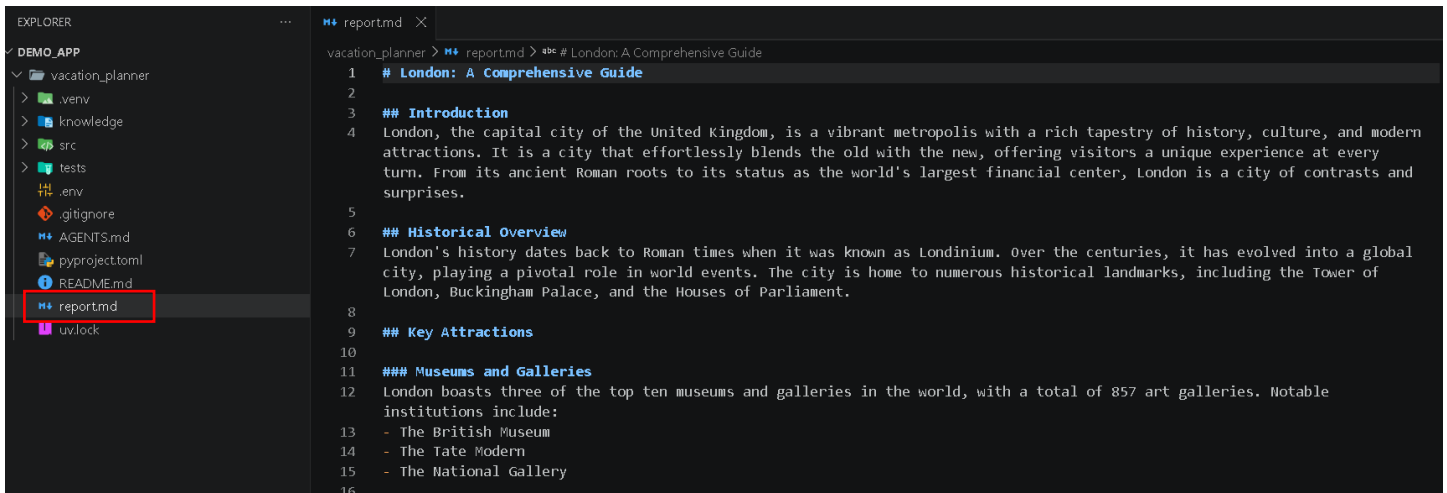
London is home to four UNESCO World Heritage sites:

1. **Tower of London** - A historic castle located on the north bank of the River Thames.
2. **Westminster** - Includes Westminster Abbey, the Houses of Parliament, and Saint Margaret's Church.
3. **Greenwich** - Known for the Royal Observatory.
4. **Maritime Greenwich** - Features the Queen's House, the Royal Observatory, and the National Maritime Museum.

Iconic Landmarks

- **Tower Bridge** - A combined bascule and suspension bridge built between 1886 and 1894.
- **Buckingham Palace** - The London residence and administrative headquarters of the monarch of the United Kingdom.
- **London Eye** - A giant Ferris wheel situated on the South Bank of the River Thames, offering panoramic views of the city.

- And now this is the Second agent result.
- And then you scroll down you will see crew completion with all result.
- Now our both agent is working perfectly.



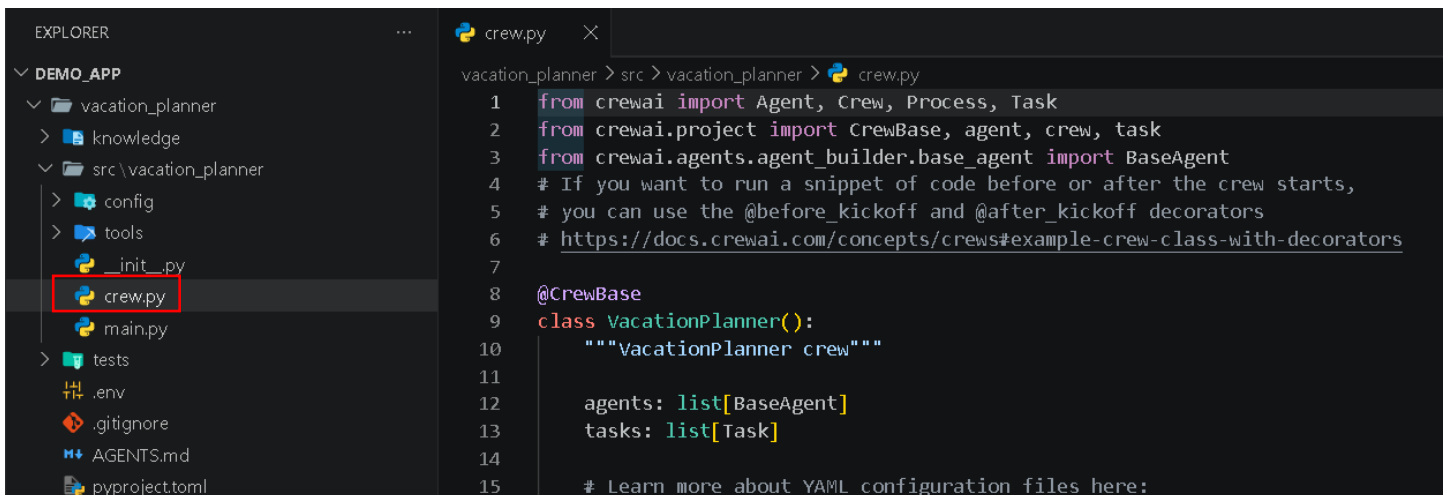
```
EXPLORER
DEMO_APP
├── vacation_planner
│   ├── .venv
│   ├── knowledge
│   ├── src
│   ├── tests
│   ├── .env
│   ├── .gitignore
│   ├── AGENTS.md
│   ├── pyproject.toml
│   ├── README.md
│   └── report.md
└── uv.lock

report.md
vacation_planner > report.md > # # London: A Comprehensive Guide
1 # London: A Comprehensive Guide
2
3 ## Introduction
4 London, the capital city of the United Kingdom, is a vibrant metropolis with a rich tapestry of history, culture, and modern attractions. It is a city that effortlessly blends the old with the new, offering visitors a unique experience at every turn. From its ancient Roman roots to its status as the world's largest financial center, London is a city of contrasts and surprises.
5
6 ## Historical Overview
7 London's history dates back to Roman times when it was known as Londinium. Over the centuries, it has evolved into a global city, playing a pivotal role in world events. The city is home to numerous historical landmarks, including the Tower of London, Buckingham Palace, and the Houses of Parliament.
8
9 ## Key Attractions
10
11 ### Museums and Galleries
12 London boasts three of the top ten museums and galleries in the world, with a total of 857 art galleries. Notable institutions include:
13 - The British Museum
14 - The Tate Modern
15 - The National Gallery
16
```

- Now in our file one report.md is created.
- Go and open it .
- Then you see there all details about that place you want to plan vacation are saved.

Frontend: -

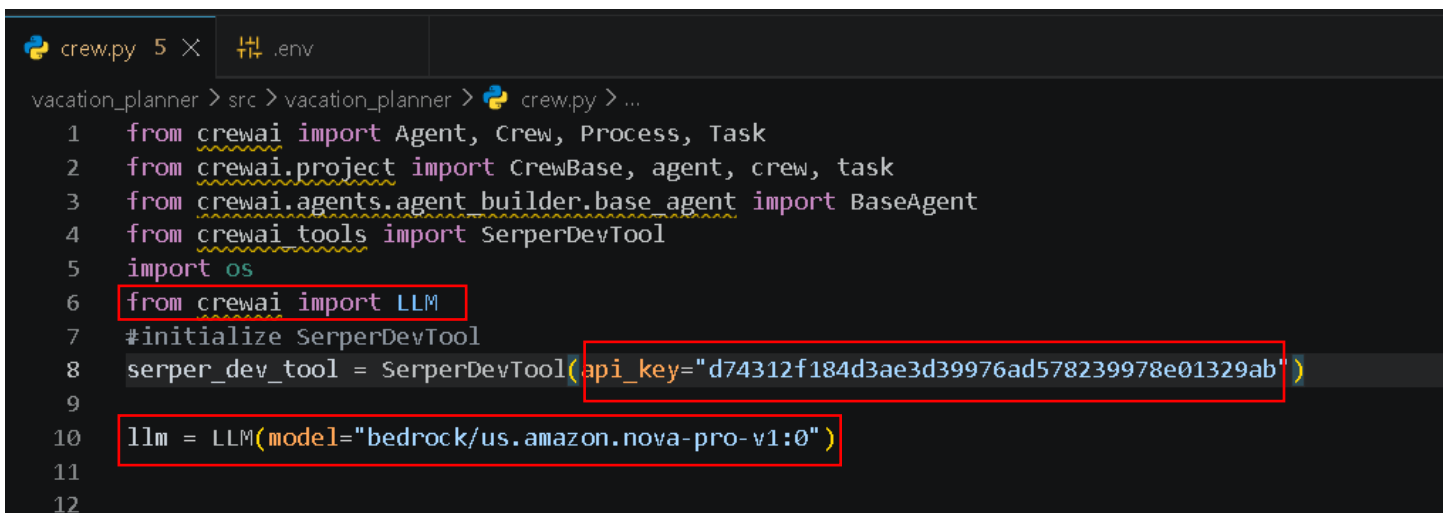
- Now for Frontend we need to add one more thing in our code.
- Let's See.



```
EXPLORER
DEMO_APP
├── vacation_planner
│   ├── knowledge
│   └── src
│       └── vacation_planner
│           ├── config
│           ├── tools
│           ├── init.py
│           ├── crew.py
│           ├── main.py
│           ├── tests
│           ├── .env
│           ├── .gitignore
│           ├── AGENTS.md
│           └── pyproject.toml
└── ...

crew.py
vacation_planner > src > vacation_planner > crew.py
1 from crewai import Agent, Crew, Process, Task
2 from crewai.project import CrewBase, agent, crew, task
3 from crewai.agents.agent_builder.base_agent import BaseAgent
4 # If you want to run a snippet of code before or after the crew starts,
5 # you can use the @before_kickoff and @after_kickoff decorators
6 # https://docs.crewai.com/concepts/crews#example-crew-class-with-decorators
7
8 @CrewBase
9 class VacationPlanner():
10     """VacationPlanner crew"""
11
12     agents: list[BaseAgent]
13     tasks: list[Task]
14
15     # Learn more about YAML configuration files here:
```

- Open crew.py inside src vacation_planner folder.



```
crew.py 5 X .env
vacation_planner > src > vacation_planner > crew.py > ...
1 from crewai import Agent, Crew, Process, Task
2 from crewai.project import CrewBase, agent, crew, task
3 from crewai.agents.agent_builder.base_agent import BaseAgent
4 from crewai.tools import SerperDevTool
5 import os
6 from crewai import LLM
7 # initialize SerperDevTool
8 serper_dev_tool = SerperDevTool(api_key="d74312f184d3ae3d39976ad578239978e01329ab")
9
10 llm = LLM(model="bedrock/us.amazon.nova-pro-v1:0")
11
12
```

- Import LLM by “ from crewai import LLM ”.
- In serper_dev_tool that you write previously.
- Now replace api_key with default api key value that you get from serper Dev.

- Now create llm by this code.
- “llm = LLM(model= “paste from .env file model” ”

```

30     @agent
31     def vacation_researcher(self) -> Agent:
32         return Agent(
33             config=self.agents_config['vacation_researcher'], # type: ignore[index]
34             verbose=True,
35             tools=[serper_dev_tool],
36             llm=llm
37         )
38
39     @agent
40     def itinerary_planner(self) -> Agent:
41         return Agent(
42             config=self.agents_config['itinerary_planner'], # type: ignore[index]
43             verbose=True,
44             llm=llm
45         )

```

- Now in both agent decorators.
- add llm = llm.

```

streamlitui.py
1  import streamlit as st
2  import os
3  from datetime import datetime
4  from src.vacation_planner.crew import VacationPlanner # Replace path 1
5
6  st.set_page_config(page_title="Personal AI Vacation Planner ", page_icon="✈️")
7
8  # Enhanced CSS styling
9  st.markdown("""
10 <style>
11 .main-header {
12     background: linear-gradient(90deg, #FF6B6B, #4ECDC4, #45B7D1, #96CEB4);
13     -webkit-background-clip: text;
14     -webkit-text-fill-color: transparent;
15     font-size: 3rem;
16     font-weight: bold;
17     text-align: center;
18     margin-bottom: 0;
19 }

```

- Now add this streamlitui.py in this folder (provided in resource).
- Now we need to install streamlit
- Open terminal and write code “uv add streamlit”.
- It takes time.

```

PS C:\Users\Akash Kumar\Documents\Demo_app> cd vacation_planner
PS C:\Users\Akash Kumar\Documents\Demo_app\vacation_planner> uv add streamlit
Resolved 168 packages in 4.69s
Built vacation-planner @ file:///C:/Users/Akash%20Kumar/Documents/Demo_app/vacation_planner
Prepared 15 packages in 19.03s
Uninstalled 1 package in 1.41s
Installed 15 packages in 16.48s
+ altair==6.1.0
+ blinker==1.9.0
+ cachetools==7.0.6
+ gitdb==4.0.12
+ gitpython==3.1.47
+ narwhals==2.20.0
+ pandas==3.0.2
+ pydeck==0.9.2
+ s3map==5.0.3
+ streamlit==1.56.0
+ tonl==0.10.2
+ tornado==6.5.5
+ tzdata==2026.2
~ vacation-planner==0.1.0 (from file:///C:/Users/Akash%20Kumar/Documents/Demo_app/vacation_planner)
+ watchdog==6.0.0
PS C:\Users\Akash Kumar\Documents\Demo_app\vacation_planner>

```

- Streamlit is installed
- Now simply run your streamlit.py file

```

PS C:\Users\Akash Kumar\Documents\Demo_app\vacation_planner> .\.venv\Scripts\activate
(vacation-planner) PS C:\Users\Akash Kumar\Documents\Demo_app\vacation_planner> pip install crewai crewai-tools streamlit
Collecting crewai
  Using cached crewai-1.14.3-py3-none-any.whl.metadata (36 kB)
Collecting crewai-tools
  Using cached crewai_tools-1.14.3-py3-none-any.whl.metadata (11 kB)
Collecting streamlit
  Using cached streamlit-1.56.0-py3-none-any.whl.metadata (9.8 kB)
Collecting aiofiles~=24.1.0 (from crewai)
  Using cached aiofiles-24.1.0-py3-none-any.whl.metadata (10 kB)
Collectingaiosqlite~=0.21.0 (from crewai)
  Using cached aiosqlite-0.21.0-py3-none-any.whl.metadata (4.8 kB)

```

- Now activate virtual environment by “`.\.venv\Scripts\activate`”
- Then install crewai tools and streamlit.

```

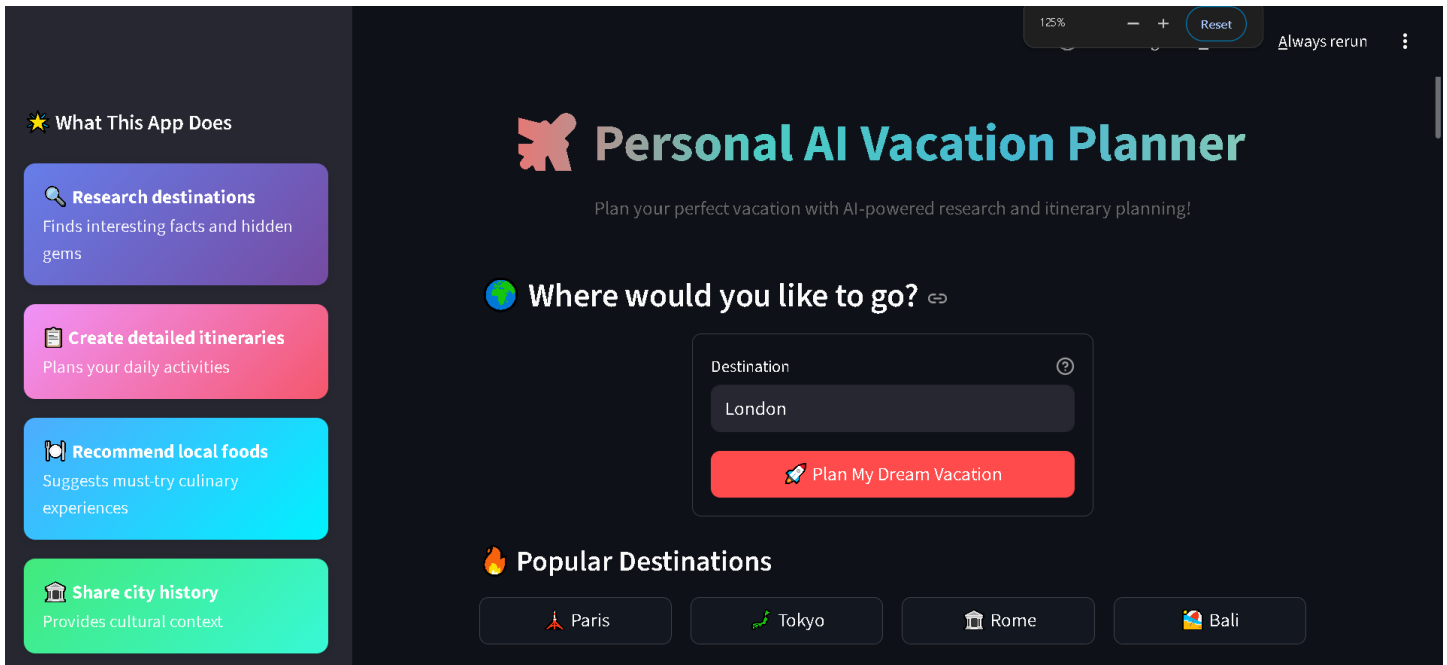
(vacation-planner) PS C:\Users\Akash Kumar\Documents\Demo_app\vacation_planner> python -m streamlit run streamlitui.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.31.238:8501

```

- Now run this by “`python -m streamlit run streamlitui.py`”.
- Now open local URL in Browser.



- Now see its working perfectly.



Your London Adventure Plan

Detailed Report about London

Introduction

London, the capital city of the United Kingdom, is a vibrant metropolis renowned for its rich history, cultural diversity, and iconic landmarks. With a population of over 9 million, it stands as one of the most cosmopolitan cities in the world.

Historical Background

- **Roman Foundation:** London was founded by the Romans around 50 AD and was initially called Londinium. They built a bridge across the River Thames, establishing it as a significant trading hub.
- **Celtic Origin:** The name London is derived from the Celtic word "Londínios," meaning "the place of the bold one."

- And gives results.